GOSIM

GOSIM Paris 2026

GOSIM Paris 2026

May 5-6, 2026 Station F, Paris

GOSIM

GOSIM Paris 2026

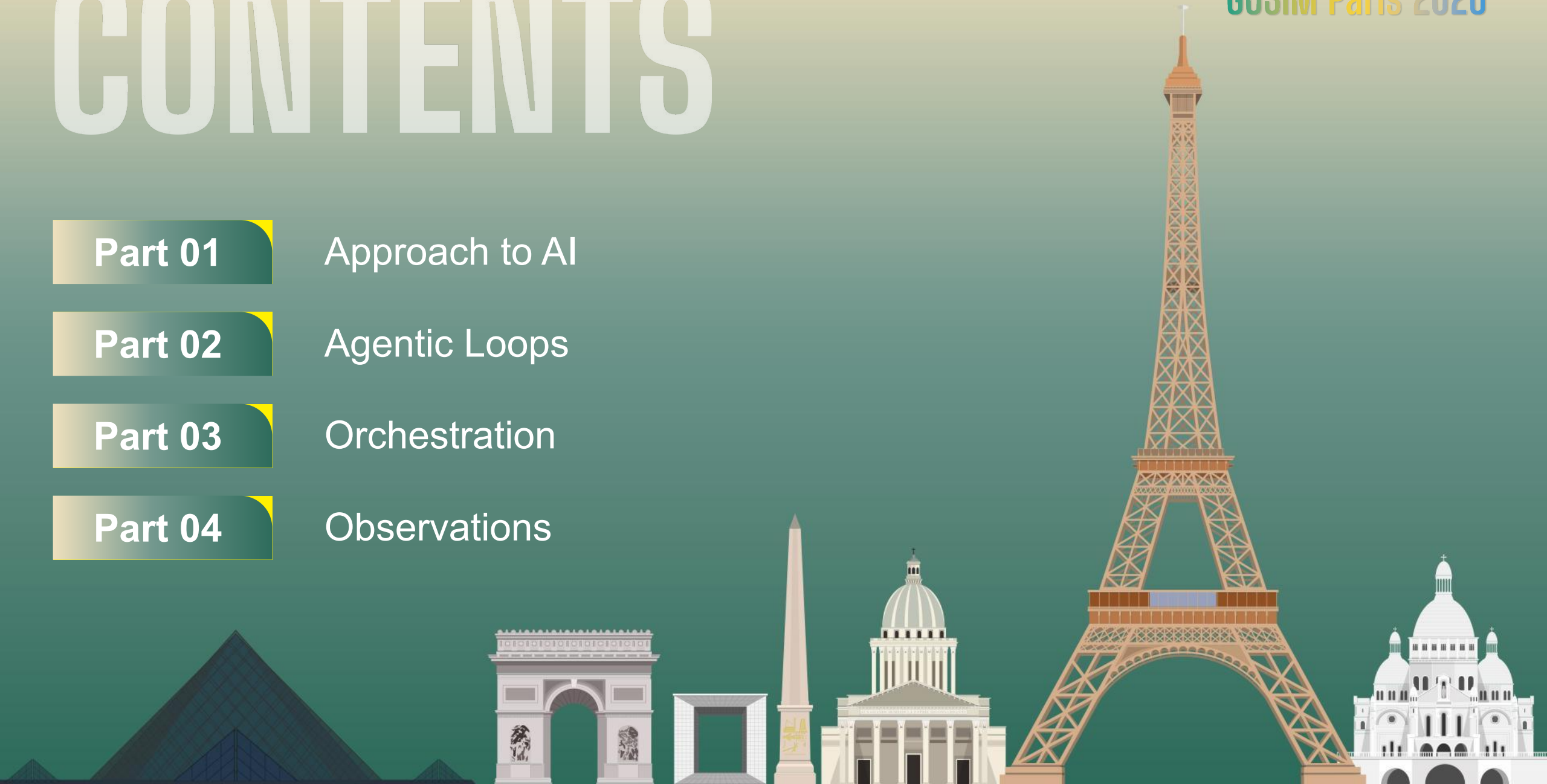
Stop looking at the code

Software Factory story

Marcin Zubrzycki strongdm ∈ Delinea

CONTENTS

GOSIM Paris 2026



Why should you care?

Built and released:

driver for google spanner in 1 week (down from 2 months)

PBAC protocol-aware driver for mssql in 2 weeks (down from 4 months)

Also:
Built an academic language compiler in 1 prompt (down from a couple days)

In general, one shots every academic assignment at my uni (WrocTech)

01 Approach to AI

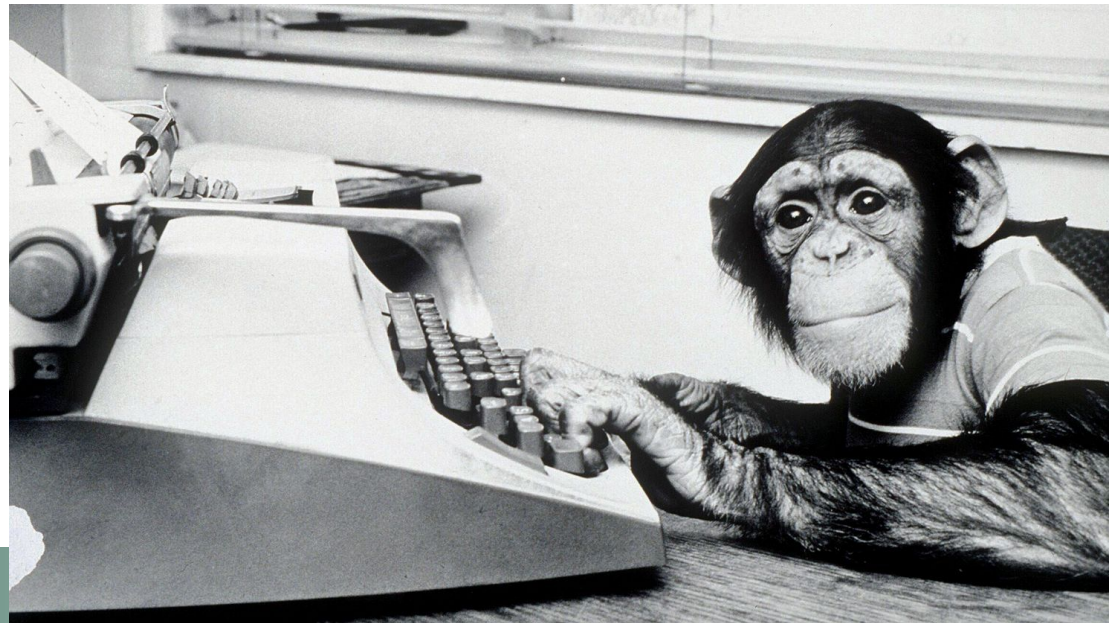
- If you didnt write it, you cant test it. You also cant document it

- Dont mix human and agent communication

Agents write to agents they vibe better than agents writing to humans (be it code, docs, or gossip)

Agent thrives when you let it own the entire repo, all unnecessary abstractions it likes is its taste

- Given appropriate constraints, modern agents given enough time compound correctness



What's still hard? - Ideation

What do you want to build?

The agent can't tell a good idea from a bad one.
Frankly it can't even tell a bug apart from a feature. god only knows what protocol should be spoken here.

The human business context in which your software exists matters. Whats good all depends on the customer you know

Marketing is not solved!

Keep yourself out of the loop containing bulk work.

Be there at the start to pick out the direction - Agents dont know *what* you want
Then have them *do* it

You dont want to be writing every converter, resolver but you should spend some time thinking about what the requirements are.

> the agent is unstoppable at do. It is hopeless at what.

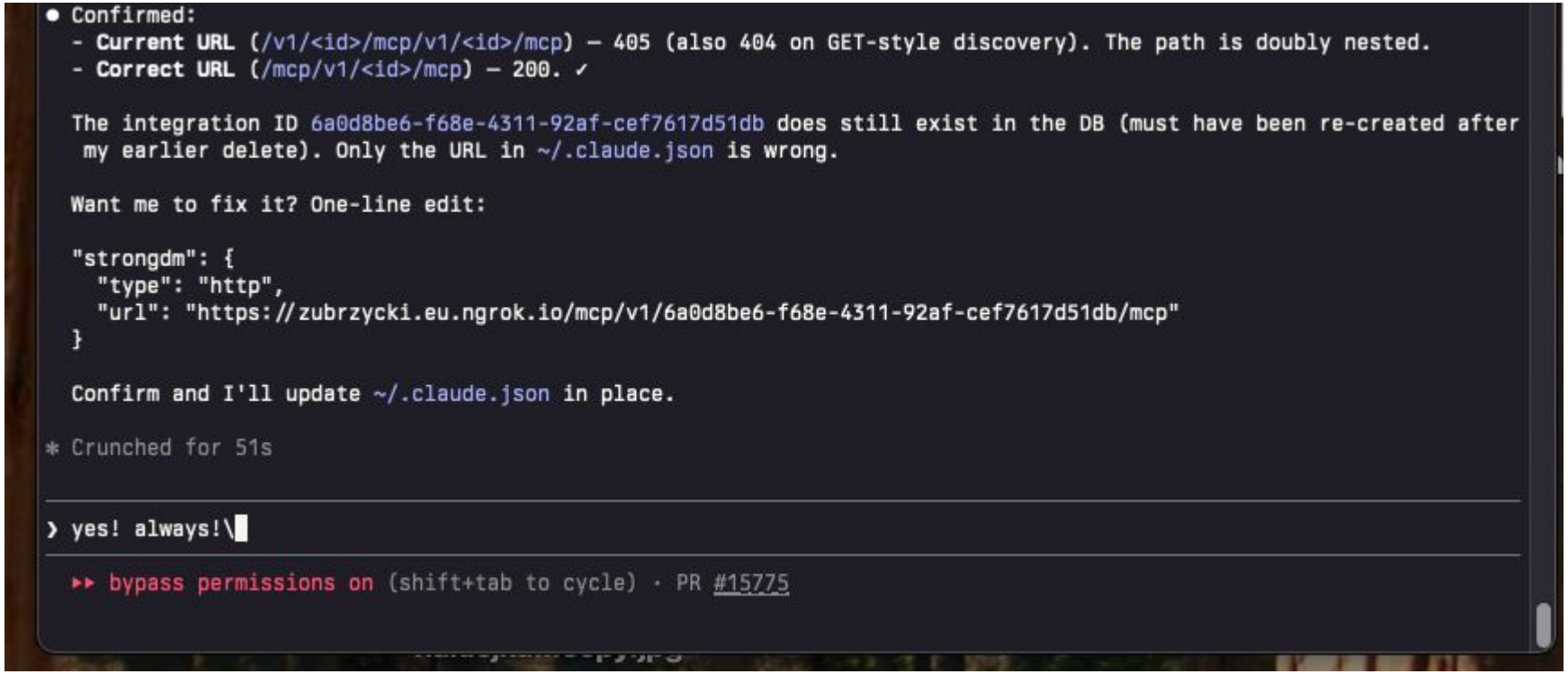
enter non interactive development

for more, visit factory.strongdm.ai

Things you shouldnt be doing

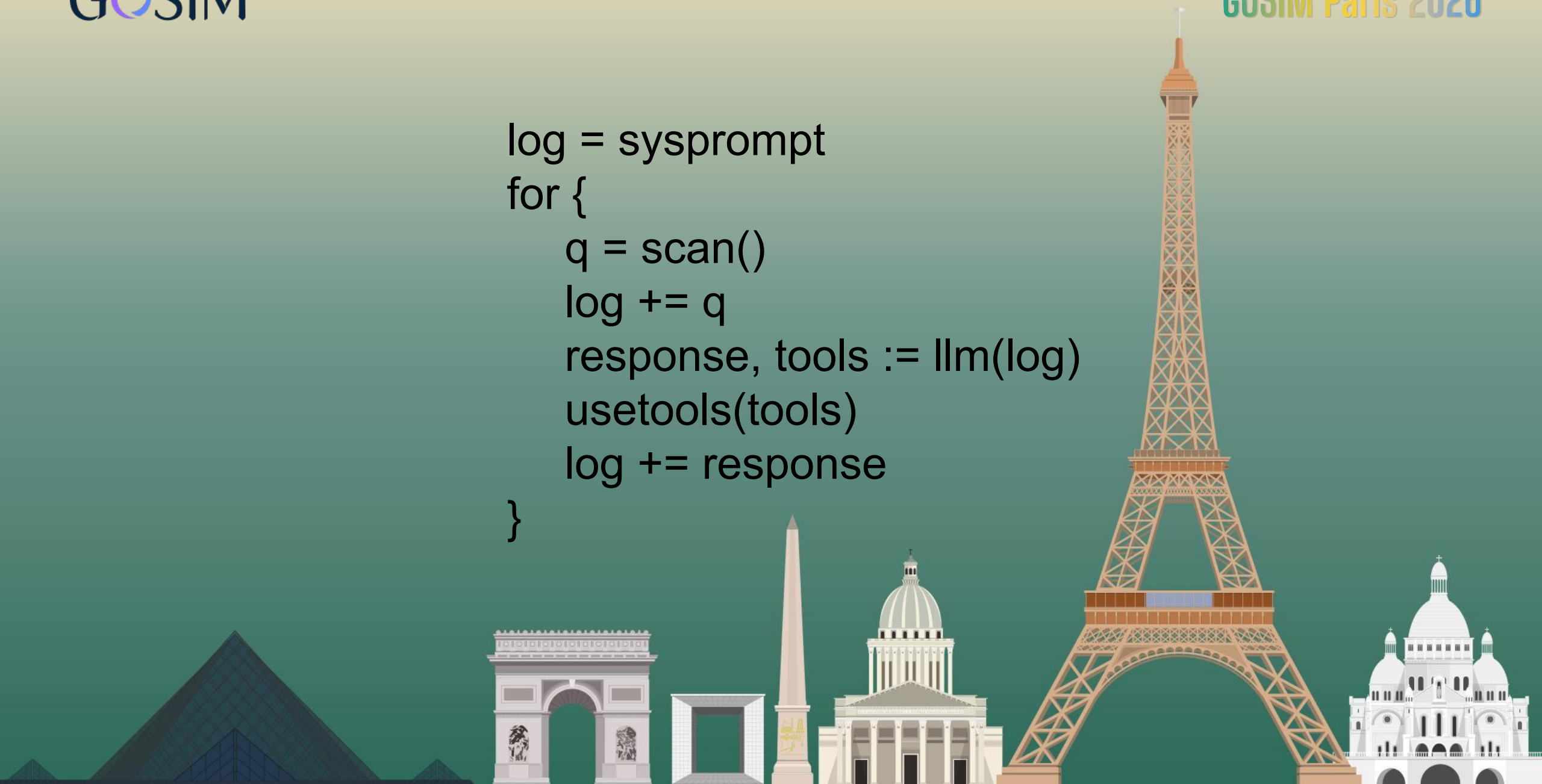
- Copy and paste messages between agents
- Answering the agent's clarifying questions mid-build
- Reading the code to decide if it’s done
- Gathering and indexing research data from the web
- **If you can read the tokens, the model can act upon them**

Make them act upon the tokens they can read



02 Agentic Loops

```
log = sysprompt
for {
    q = scan()
    log += q
    response, tools := llm(log)
    usetools(tools)
    log += response
}
```



Classic software tries to pin down inputs, outputs, and edge cases up front. Agentic loops loosen that up. The system can react to what it sees. The constraints move elsewhere: the user's intent, the tools on hand, and the quality of feedback.

https://web.navan.dev

Truth be told, implementation is easy once you have a detailed specification.
The hard part of building is design

The builder shouldnt be answering questions
Dont mix human and ai code, dont fight its implementation decisions

Treat the loop like a person

What can it see? What can it change?

How can it tell whether it succeeded?

Build with tools, convert data into tokens

What could it see and interact with?

- full compiler output & a vm
- logs
- screenshots
- traces
- diffs
- HTTP responses
- database state
- browser console output
- previous failed attempts
- flaky behavior over time
- related incidents
- recently changed files

How can you trust code you haven’t read?
You can’t TRUST anyone!

You need a validator which will make sure the system can do everything you need from it.

We assume a if it walks like a duck and looks like a duck it is a duck philosophy

Tests++ -> **Scenario**

A scenario represents a user story, and we have agents empirically confirm the users can do exactly everything they should

Design your apps for agent use through CLI, run simulations where you have agents confirm everything the users can do is working.
You are fighting against reward hacking, have the scenarios deliver proof of work

- Want to always support screen readers for new features? Add some agents using screen readers and make them too perform all scenarios
- Driver development? give Server, client, driver, have it compare results
- Building a compiler? Have agents compile and run the programs and compare outputs
- Porting to a new SDK? Give several clients and the server

Validator for Integrations: Digital Twin

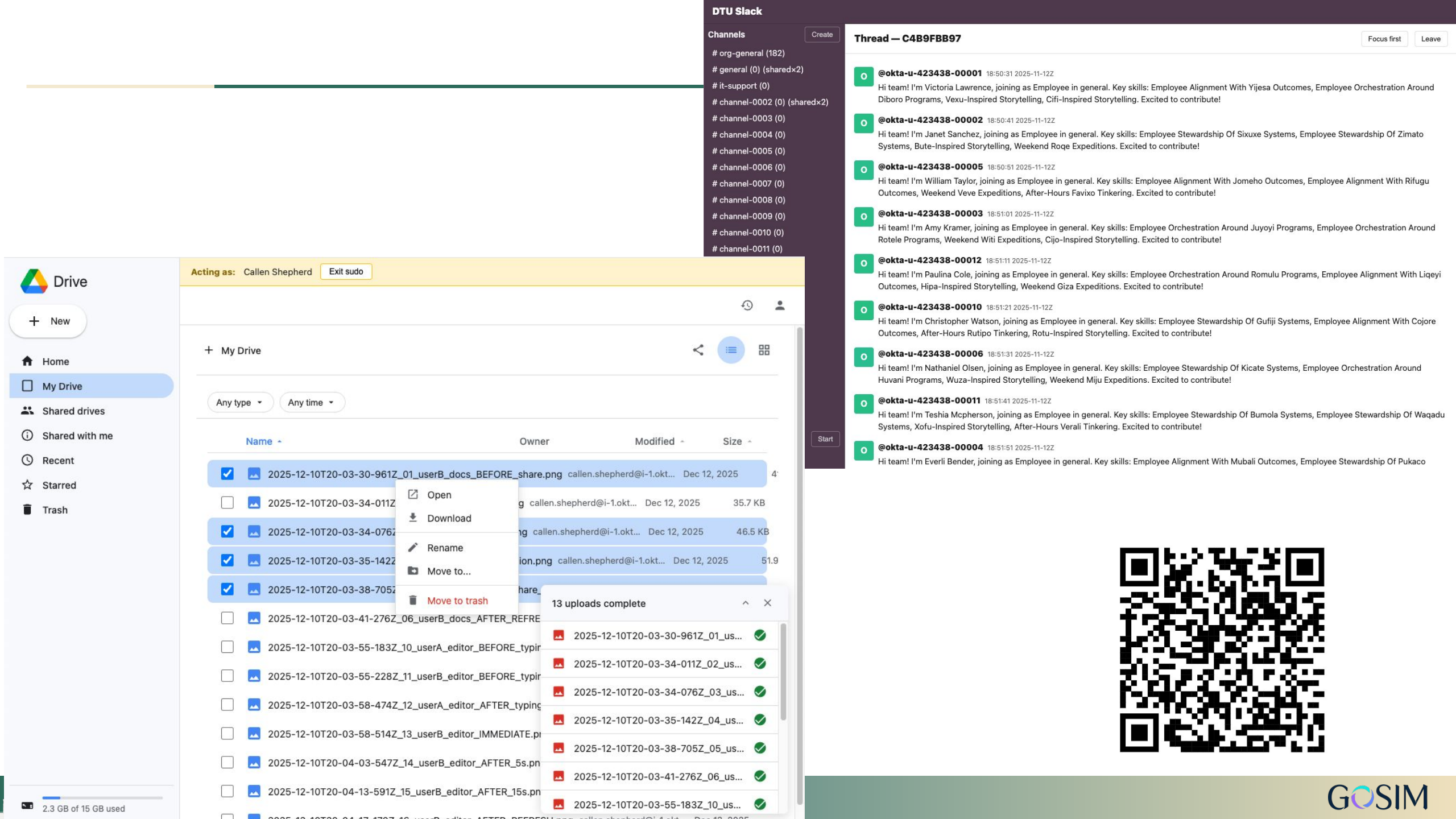
Have the agents build themselves an environment as close as possible to the real deal

Build a Digital Twin of the service you're integrating with
Gather wire data from real clients and "semantically port" it
Mocks++

Don't need 100% fidelity, be good enough for official clients to work

Then you can customize the behavior, spin up instances, not care for API costs & rate limits

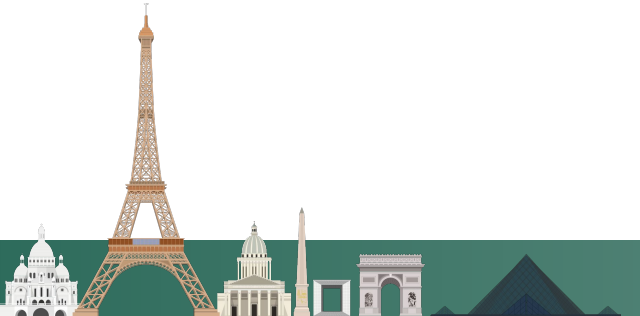
Such mocks were always desirable,
but unfeasible economically


DTU Slack
Channels
Create
org-general (182)
general (0) (shared×2)
it-support (0)
channel-0002 (0) (shared×2)
channel-0003 (0)
channel-0004 (0)
channel-0005 (0)
channel-0006 (0)
channel-0007 (0)
channel-0008 (0)
channel-0009 (0)
channel-0010 (0)
channel-0011 (0)
Thread — C4B9FBB97
Focus first
Leave
@okta-u-423438-00001 18:50:31 2025-11-12Z
Hi team! I'm Victoria Lawrence, joining as Employee in general. Key skills: Employee Alignment With Yijesa Outcomes, Employee Orchestration Around Diboro Programs, Vexu-Inspired Storytelling, Cifi-Inspired Storytelling. Excited to contribute!
@okta-u-423438-00002 18:50:41 2025-11-12Z
Hi team! I'm Janet Sanchez, joining as Employee in general. Key skills: Employee Stewardship Of Sixuxe Systems, Employee Stewardship Of Zimato Systems, Bute-Inspired Storytelling, Weekend Roqe Expeditions. Excited to contribute!
@okta-u-423438-00005 18:50:51 2025-11-12Z
Hi team! I'm William Taylor, joining as Employee in general. Key skills: Employee Alignment With Jomeho Outcomes, Employee Alignment With Rifugu Outcomes, Weekend Veve Expeditions, After-Hours Favixo Tinkering. Excited to contribute!
@okta-u-423438-00003 18:51:01 2025-11-12Z
Hi team! I'm Amy Kramer, joining as Employee in general. Key skills: Employee Orchestration Around Juyoyi Programs, Employee Orchestration Around Rotele Programs, Weekend Witi Expeditions, Cijo-Inspired Storytelling. Excited to contribute!
@okta-u-423438-00012 18:51:11 2025-11-12Z
Hi team! I'm Paulina Cole, joining as Employee in general. Key skills: Employee Orchestration Around Romulu Programs, Employee Alignment With Liqeyi Outcomes, Hipa-Inspired Storytelling, Weekend Giza Expeditions. Excited to contribute!
@okta-u-423438-00010 18:51:21 2025-11-12Z
Hi team! I'm Christopher Watson, joining as Employee in general. Key skills: Employee Stewardship Of Gufiji Systems, Employee Alignment With Cojore Outcomes, After-Hours Rutipo Tinkering, Rotu-Inspired Storytelling. Excited to contribute!
@okta-u-423438-00006 18:51:31 2025-11-12Z
Hi team! I'm Nathaniel Olsen, joining as Employee in general. Key skills: Employee Stewardship Of Kicate Systems, Employee Orchestration Around Huvani Programs, Wuza-Inspired Storytelling, Weekend Miju Expeditions. Excited to contribute!
@okta-u-423438-00011 18:51:41 2025-11-12Z
Hi team! I'm Teshia Mcpherson, joining as Employee in general. Key skills: Employee Stewardship Of Bumola Systems, Employee Stewardship Of Waqadu Systems, Xofu-Inspired Storytelling, After-Hours Verali Tinkering. Excited to contribute!
@okta-u-423438-00004 18:51:51 2025-11-12Z
Hi team! I'm Everli Bender, joining as Employee in general. Key skills: Employee Alignment With Mubali Outcomes, Employee Stewardship Of Pukaco
Start
Drive
Acting as: Callen Shepherd
Exit sudo
New
Home
My Drive
Shared drives
Shared with me
Recent
Starred
Trash
2.3 GB of 15 GB used
My Drive
Any type
Any time
Name
Owner
Modified
Size
2025-12-10T20-03-30-961Z_01_userB_docs_BEFORE_share.png callen.shepherd@i-1.okt... Dec 12, 2025
2025-12-10T20-03-34-011Z callen.shepherd@i-1.okt... Dec 12, 2025 35.7 KB
2025-12-10T20-03-34-076Z callen.shepherd@i-1.okt... Dec 12, 2025 46.5 KB
2025-12-10T20-03-35-142Z callen.shepherd@i-1.okt... Dec 12, 2025
2025-12-10T20-03-38-705Z
2025-12-10T20-03-41-276Z_06_userB_docs_AFTER_REFRE
2025-12-10T20-03-55-183Z_10_userA_editor_BEFORE_typir
2025-12-10T20-03-55-228Z_11_userB_editor_BEFORE_typir
2025-12-10T20-03-58-474Z_12_userA_editor_AFTER_typing
2025-12-10T20-03-58-514Z_13_userB_editor_IMMEDIATE.pr
2025-12-10T20-04-03-547Z_14_userB_editor_AFTER_5s.pn
2025-12-10T20-04-13-591Z_15_userB_editor_AFTER_15s.pn
Open
Download
Rename
Move to...
Move to trash
13 uploads complete
2025-12-10T20-03-30-961Z_01_us...
2025-12-10T20-03-34-011Z_02_us...
2025-12-10T20-03-34-076Z_03_us...
2025-12-10T20-03-35-142Z_04_us...
2025-12-10T20-03-38-705Z_05_us...
2025-12-10T20-03-41-276Z_06_us...
2025-12-10T20-03-55-183Z_10_us...

The hard part: Validation

Validator is what builds confidence in the product

Driver development example
The builder is not allowed to say “the driver works.”
The validator has to prove:
✅ connects
✅ authenticates
✅ runs queries
✅ preserves types
✅ handles errors
✅ survives bad input
✅ matches official client behavior



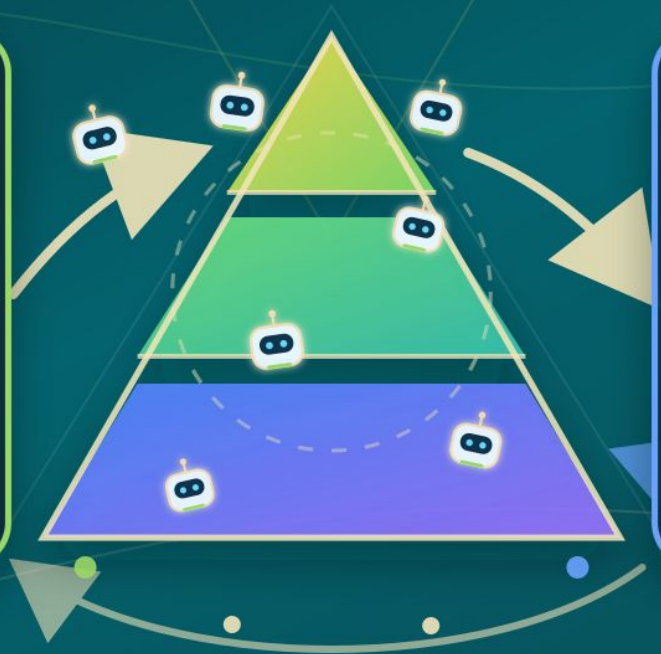
03

Orchestration

THE LOOP

1 VALIDATION

Your validation harness must be end-to-end, as close to the real environment as possible: customers, integrations, economics.



2 FEEDBACK

A sample of the output, fed back into the inputs. This closed loop allows your system to self-correct and compound correctness.

The loop runs until the holdout scenarios pass (and stay passing)

Attractor is claude code+

Just like claude uses LLMs started over and over Attractor spawns and kills agents

its agentic os, programs call each other by llm inference

Definition of done machine
Abstracts away the implementation

Attractor is glue: it eliminates yourself as the bus off messages between agents.

It consumes a prompt in the form of a dotfile like skill for claude code

ATTRACTOR: AGENTIC OS

GOSIM Paris 2026

CONSUMES A DOTFILE

Workflow defined as a graph

workflow.dot

```
digraph workflow {
  start -> plan;
  plan -> build;
  build -> validate;
  validate -> document;
  document -> validate;
  validate -> done;
}
```

EACH EDGE HAS A PROMPT

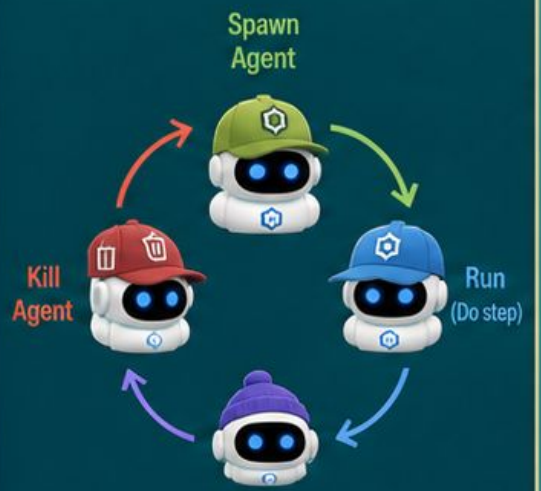
from — [prompt] → to

ATTRACTOR (CLAUDE CODE+)

Agentic OS

ATTRACTOR

Definition of Done Machine



Spawns and kills agents over and over.

AGENTS CALL EACH OTHER BY PROMPTING

Programs (agents) prompt other programs to get work done.

ABSTRACTION

Abstracts away implementation

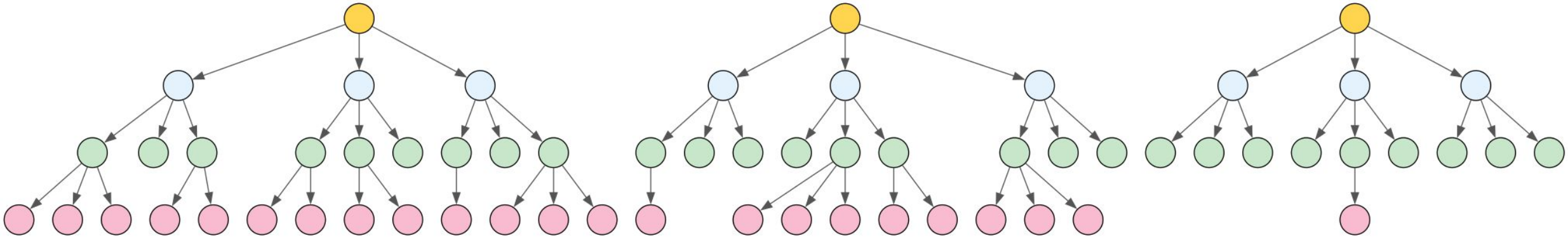
You define **WHAT** and **WHEN.**

Attractor handles **HOW.**

You can’t store everything in context,
There’s too many things to remember exactly

Add a feature to your documentation: more/less info button

index the documentation of your systems
docs/L1-{system/}/{info1.md, info2.md, …, L2-{subsystem/}/…



MANAGE your context in real time.
SPAWN A NEW CLAUDE with perfectly zoomed context for every step of the way

Also index your inspirations/

DOCUMENTATION PYRAMID SUMMARIES FLOW

GOSIM Paris 2026

Agent gets the right level of context for the task through the docs directory

DOCUMENTATION PYRAMID

From overview to details

More context
Higher level

1. SUMMARIES
Why & what
Big picture decisions

2. GUIDES
How to
Workflows & patterns

3. REFERENCE
What is
Technical reference

4. DETAILS
Deep dive
In-depth explanations

5. SOURCE
Raw truth
Code, schemas, configs

Less context
Lower level

THROUGH THE DOCS DIRECTORY

Structured for the agent

```
docs/
  00-summaries/
    README.md
  10-guides/
    ...
    setup.md
    deploy.md
  20-reference/
    ...
    api.md
    data-model.md
  30-details/
    ...
    architecture.md
    security.md
  40-source/
    ...
    schema.sql
    config.yaml
```

READS RELEVANT DOCS

AGENT

Gets the right context for the step

Context window
(just right)

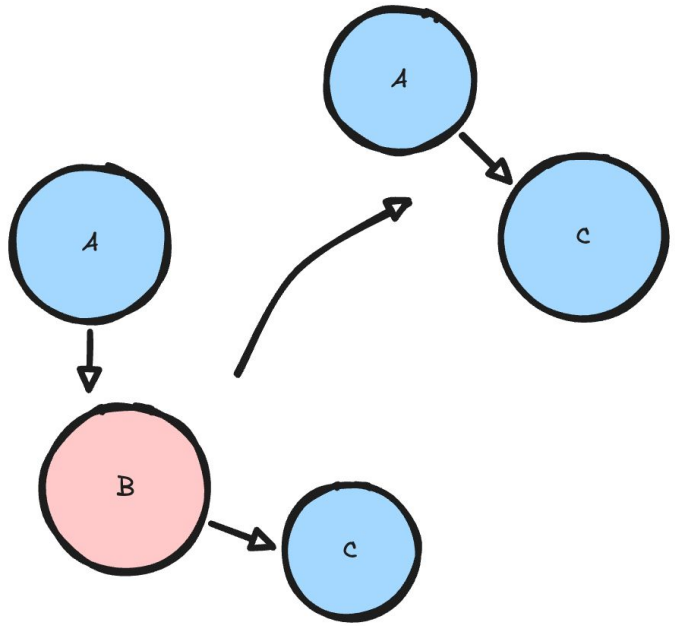
- Right level
- Focused
- Relevant
- Up-to-date

THE RESEARCHER
Helps you figure out what you want, have agents compile you a base of all available research up to this point, give you runnable experiments, download all existing implementations etc
inspirations/ folder

THE SPECIALIST
Different providers do different things better: Gemini can search google, Grok has real time data, claude and codex are unstoppable. Have all of them think about your problem and synthesize a document

THE MAINTAINER
Runs all CI Jobs after done building
solve merge conflicts when updating libraries

THE LUMBERJACK
Finds useless relay work in the
middle, cuts it out



Gear up for your campaign! Generate the dotfile

Agentic loops are your roles

THE BUILDER

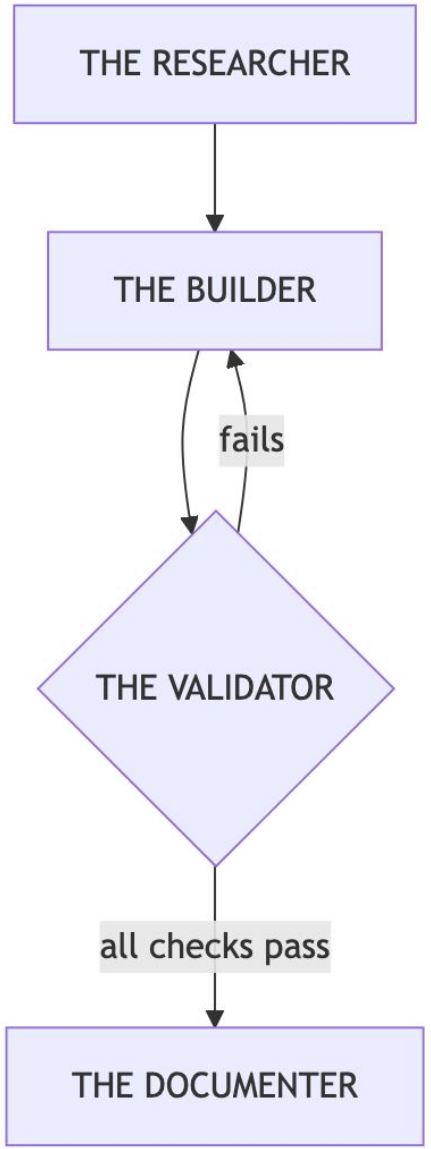
THE VALIDATOR
THE DOCUMENTER

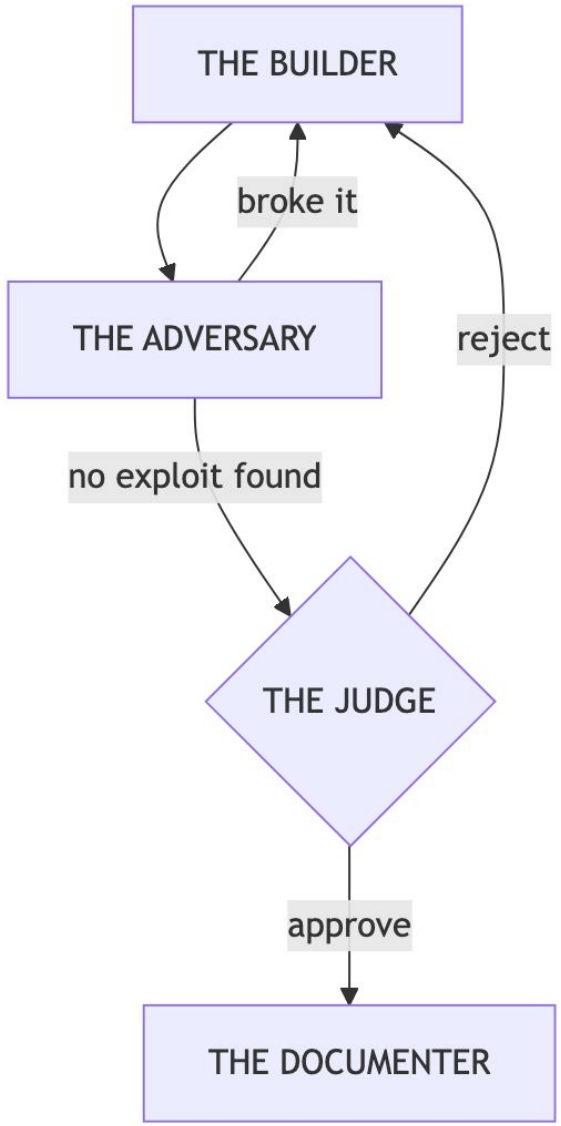
Advanced roles:
THE INTERWIEVER
THE RESEARCHER
THE PLANNER
[...]

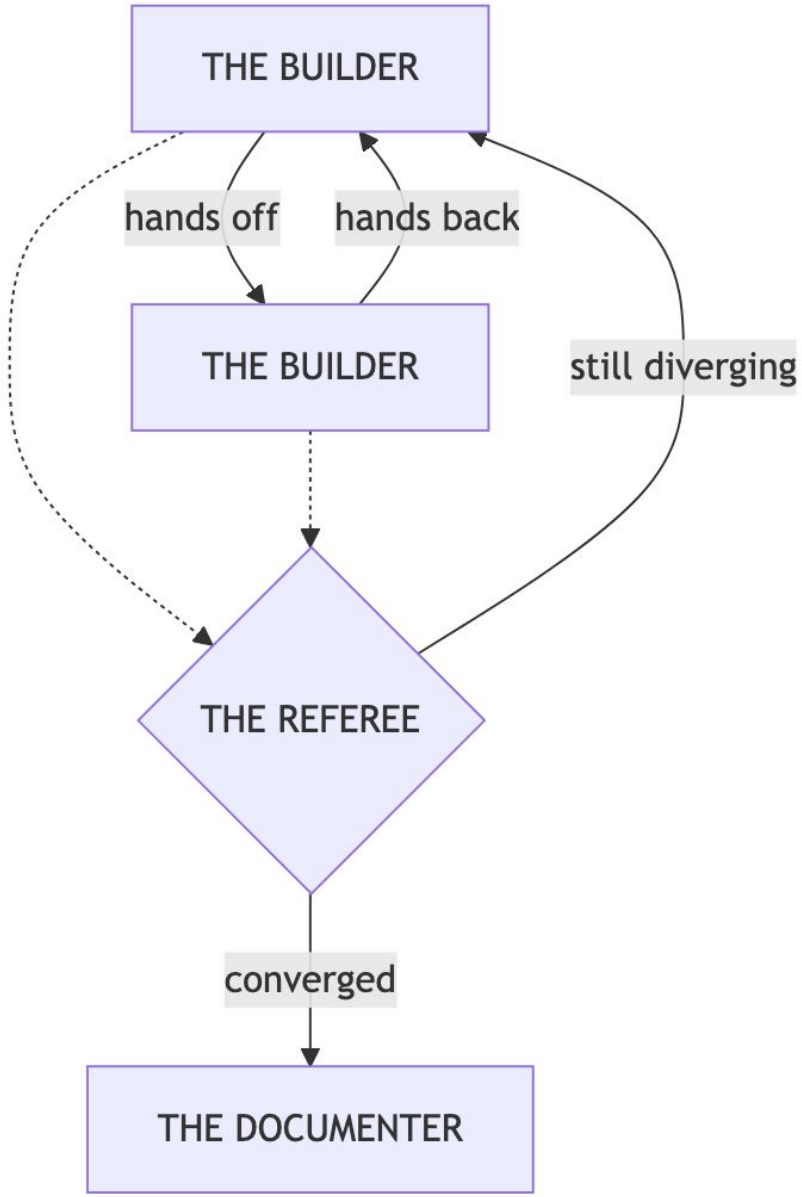
Program with them! Combine and mix & match

THE USUAL FLOW

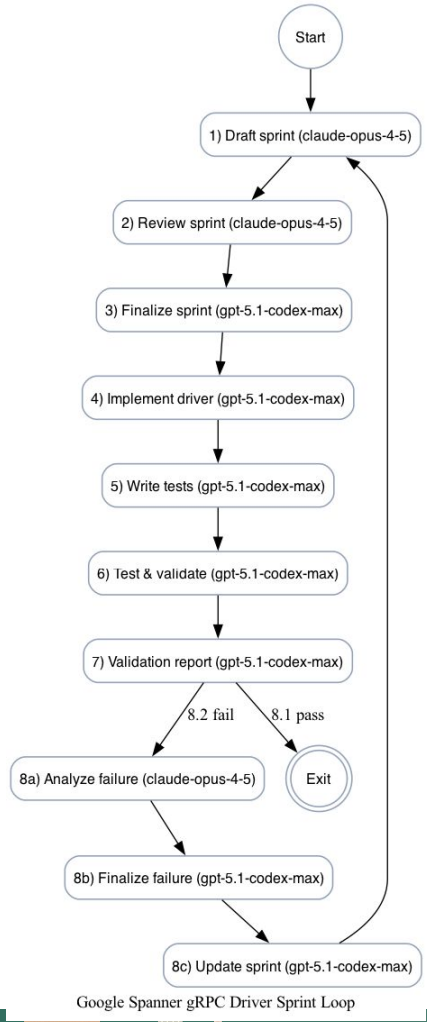
0. Research it
1. Build it
2. Validate it
3. Judge it
4. Document learnings (leave traces, maybe update your plan?)


THE RESEARCHER
THE BUILDER
fails
THE VALIDATOR
all checks pass
THE DOCUMENTER


THE BUILDER
broke it
THE ADVERSARY
reject
no exploit found
THE JUDGE
approve
THE DOCUMENTER

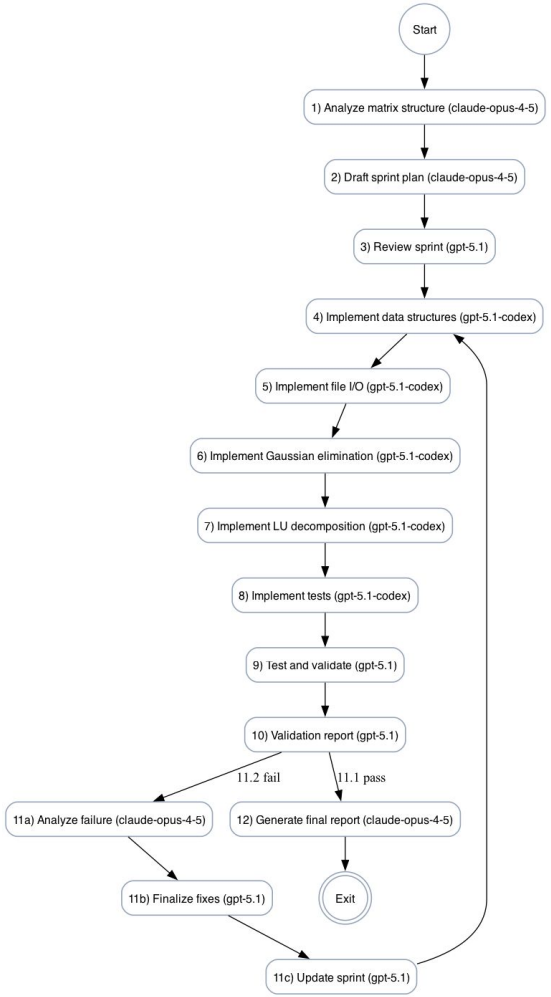

THE BUILDER
hands off
hands back
THE BUILDER
still diverging
THE REFEREE
converged
THE DOCUMENTER

develop driver

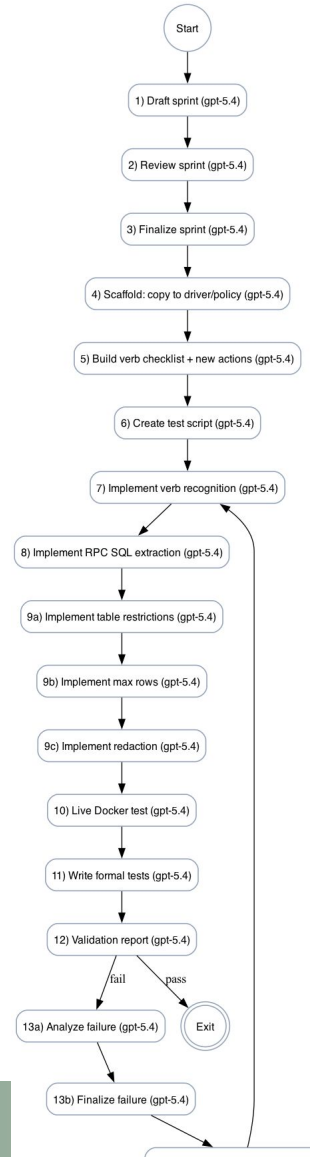


Google Spanner gRPC Driver Sprint Loop

do a computer science assignment



Julia Block Matrix Solver Sprint



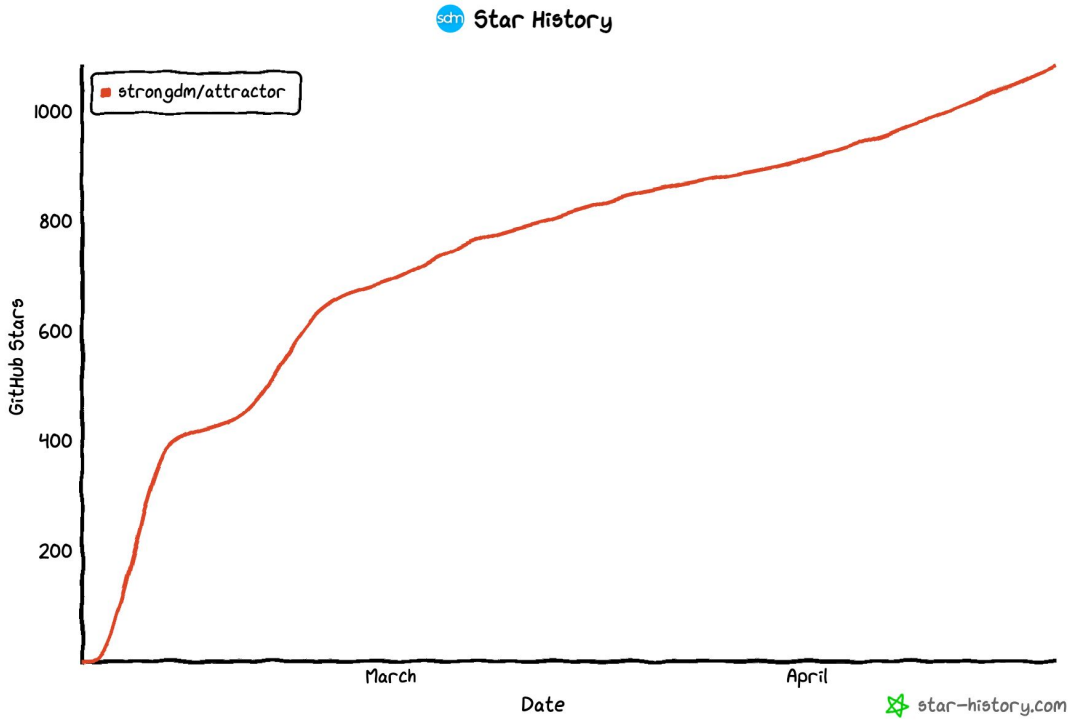
Build your own attractor

Supply the following prompt to a modern coding agent (Claude Code, Codex, OpenCode, Amp, Cursor, etc):

```
codeagent> Implement Attractor as described by https://github.com/strongdm/attractor
```

Ask yourself? Why am I doing this?
ask attractor to build the feature
(do it right now)

Grows with you!



GOSIM

GOSIM Paris 2026

What happens when you turn the knob to 11?

ATTRACTOR ARMY IN ACTION

GOSIM Paris 2026

Jira ticket → Attractor loop → Database driver → Video proof

1. JIRA TICKET

Picked up by Attractor

Jira

DRV-1472

Implement database driver

To Do

Scope

- Connect & auth
- Run queries
- Transactions
- Types & encoding
- Errors & edge cases
- Match official client behavior

Labels: driver, database, validation

Priority: High

Assignee: Unassigned

WORKFLOW (METHOD)

INPUTS

- Ticket
- Workflow

2. ATTRACTOR

Orchestrates the army

ATTRACTOR LOOP

Plan
Understand requirements, design test scenarios, choose approach

Build
Implement driver, add tests, fix issues

Validate
Run scenarios against many DB servers & versions
Observe database results
Compare with real client data

Document
Update docs, examples, usage, known limitations

Report
Summarize results, coverage, remaining gaps

3. PROOF DELIVERED

Video proof + artifacts

PROOF: DRV-1472 COMPLETED

Database driver validated across many servers

Video proof (running scenarios)

Running scenarios...

Connect & auth	PASS
Run queries	PASS
Transactions	PASS
Types & encoding	PASS
Errors & edge cases	PASS
Compare with real client	PASS

482 / 482 scenarios passed

SAMPLE RESULT (QUERY)

SELECT * FROM users

Database Result

id	name	email
1	Alice	a@ex.com
2	Bob	b@ex.com
3	Carol	c@ex.com

Real Client Result

id	name	email
1	Alice	a@ex.com
2	Bob	b@ex.com
3	Carol	c@ex.com

No differences

0:00 / 0:15

ARTIFACTS

test-report.html · query-results.csv · logs.zip · docs/

CONFIDENCE THROUGH VALIDATION

Real servers. Real queries. Real client comparison. Evidence over claims.

Many servers | Real client parity | All scenarios passed | Video proof | Evidence delivered

04 Observations

Observations

The models have been good enough for all this for a while, we aren’t waiting for better models

claude code style harness truly is the next milestone after base LLMs

It takes unreliable components into a reliable system

Real programming still happens! it shifts to writing the ticket + picking the appropriate workflow

models live on the backs of other models, in the end, its vibes all the way down.

Agents will not learn for you! You still have to educate yourself to make the best calls. Its best to be smarter than the agent

Vision is not there, for frontend work you need a brilliant human designer :)

Is it secure? You need a human to figure out all the attack vectors before release :)

The agent has no clue if the gradients look cool

Takeaways!

- Definition of done is key
- Validation strategy is key
- Build to be testable by agents
- Invest in yourself

- “Stop looking at the code?” Just use the thing when agent is done

- This works when we arent treating the code as the end product, only as a way to get to the app

- All in all, at some level, the job stays the same

GOSIM

GOSIM Paris 2026

Thanks!